grEPI

Global Repository of Epidemiological Parameters

API documentation

April 2026, v2

1. Overview

grEPI provides a public REST API that returns data in JSON format. The API is read-only (GET requests only) and does not require authentication or a Collaboratory account. There are four endpoints, each corresponding to a core data domain in grEPI.

Endpoint	Description
`EpiParameterEstimates`	Epidemiological parameter estimates extracted from the literature
`PublicArticle`	Literature records (articles, preprints) from which parameter estimates have been extracted
`PathogenTaxonomy`	Pathogen taxonomy reference data with grEPI preferred names and ICD classification codes
`DiseaseCausedbyPathogen`	Disease-to-pathogen mappings with grEPI preferred names and ICD classification codes

The interactive API documentation (Swagger UI) is available at:
https://collaboratory.who.int/grepi/api/swagger/index.html

The underlying OpenAPI specification (JSON) is available at:
https://collaboratory.who.int/grepi/api/swagger/v1/swagger.json

How requests work

All requests are HTTP GET, meaning the API is read-only and used solely for retrieving data. The base URL for all requests is:

```
https://collaboratory.who.int/grepi/api
```

Append the endpoint name (e.g. `EpiParameterEstimates`, `PublicArticle`) and any query parameters to construct a request. For example:

```
https://collaboratory.who.int/grepi/api/PublicArticle?Article_Publication_Year=2024
&Author=Miura
```

All query parameters are optional. When no parameters are provided, the endpoint returns all records. When multiple parameters are provided, they are combined using AND logic, so only records matching all specified criteria are returned.

Response format

Responses are JSON arrays. Each element is a JSON object representing one record. All fields for every record are returned on every call. Please note, the API does not currently support pagination, as a result unfiltered queries may return large result sets. Use query parameters to filter where possible.

String matching

String filters are case-insensitive and support partial matching. For example, all of the following return results for Measles:

```
Disease_Name_Preferred=Measles

Disease_Name_Preferred=measles

Disease_Name_Preferred=measle
```

Date format

Date parameters (such as `Created_Date_From` and `Last_Modified_Date_From`) accept dates in ISO 8601 format: `YYYY-MM-DDTHH:MM:SS`. For example: `2026-01-01T00:00:00`.

2. Getting Started

This section walks through common tasks using the grEPI API from R and Python. No API key or authentication is needed. All examples can be run as-is.

R (using httr2)

Your first API call

Install `{httr2}` if you haven't already, then retrieve all measles parameter estimates:

```
# install.packages("httr2")
library(httr2)


resp <- request("https://collaboratory.who.int/grepi/api/EpiParameterEstimates") |>
  req_url_query(Disease_Name_Preferred = "Measles") |>
  req_perform()


measles <- resp_body_json(resp, simplifyVector = TRUE)


# Check what came back
cat("Records:", nrow(measles), "\n")
cat("Fields:", ncol(measles), "\n")
```

Selecting columns

The API returns all fields. To work with a subset, select columns after retrieval:

```
measles[, c("grEPI_ID", "epi_Parameter_Category_Name",
            "epiParameter_Estimate_Type",
            "epiParameter_Estimate_Subtype",
            "epiParameter_Estimate_Point_Value",
            "epiParameter_Estimate_Unit")]
```

Combining filters

Retrieve only Case Fatality Ratio estimates for measles:

```
resp <- request("https://collaboratory.who.int/grepi/api/EpiParameterEstimates") |>
  req_url_query(
    Disease_Name_Preferred = "Measles",
    Estimate_Type_Name = "Case Fatality Ratio (CFR)"
  ) |>
```

```
  req_perform()


cfr <- resp_body_json(resp, simplifyVector = TRUE)
```

Searching the literature

Search articles by title keyword and publication year. The `Article_Title` filter supports partial, case-insensitive matching:

```
resp <- request("https://collaboratory.who.int/grepi/api/PublicArticle") |>
  req_url_query(Article_Title = "mpox", Article_Publication_Year = 2024) |>
  req_perform()


articles <- resp_body_json(resp, simplifyVector = TRUE)
```

Looking up an article by DOI

DOIs should be provided without the https://doi.org/ prefix:

```
resp <- request("https://collaboratory.who.int/grepi/api/PublicArticle") |>
  req_url_query(Article_DOI = "10.1016/j.vaccine.2022.02.057") |>
  req_perform()


article <- resp_body_json(resp, simplifyVector = TRUE)
```

Using reference data endpoints

Retrieve the full disease-pathogen mapping and pathogen taxonomy:

```
# Disease-pathogen mapping
resp <- request("https://collaboratory.who.int/grepi/api/DiseaseCausedbyPathogen")
|>
  req_perform()
mapping <- resp_body_json(resp, simplifyVector = TRUE)


# Pathogen taxonomy for a specific family
resp <- request("https://collaboratory.who.int/grepi/api/PathogenTaxonomy") |>
  req_url_query(Pathogen_Family_Name_grEPI_preferred = "Paramyxovirus") |>
  req_perform()
taxonomy <- resp_body_json(resp, simplifyVector = TRUE)
```

Discovering valid values

To find which diseases, estimate types, or article labels are available:

```
# Valid disease names
```

```
resp <- request("https://collaboratory.who.int/grepi/api/DiseaseCausedbyPathogen")
|>
  req_perform()
diseases <- resp_body_json(resp, simplifyVector = TRUE)
unique(diseases$disease_Name_grEPI_preferred)


# Valid estimate types for a specific disease
resp <- request("https://collaboratory.who.int/grepi/api/EpiParameterEstimates") |>
  req_url_query(Disease_Name_Preferred = "Mpox") |>
  req_perform()
mpox <- resp_body_json(resp, simplifyVector = TRUE)
unique(mpox$epiParameter_Estimate_Type)
```

Working with nested fields

Some fields contain nested data. `epi_Parameter_Population_Country_list` is an array of objects (each with `country_Name`, `country_Code_ISO2`, `country_Code_ISO3`, `country_World_Continent`, `country_WHO_Region`). `article_Authors` is a flat array of name strings. `epi_Parameter_Population_Sex_List` and `epi_Parameter_Pathogen_Transmissions_List` are also arrays.

With `{jsonlite}`'s `simplifyVector = TRUE`, nested arrays become list columns. To access them:

```
# Country data for the first record
measles$epi_Parameter_Population_Country_list[[1]]
```

Python (using requests and pandas)

Your first API call

Install `requests` and `pandas` if you haven't already, then retrieve all measles parameter estimates:

```
import requests
import pandas as pd

url = "https://collaboratory.who.int/grepi/api/EpiParameterEstimates"
params = {"Disease_Name_Preferred": "Measles"}

response = requests.get(url, params=params)
response.raise_for_status()

measles = pd.DataFrame(response.json())
print(f"Records: {len(measles)}, Fields: {len(measles.columns)}")
```

Selecting columns

```
measles[["grEPI_ID", "epi_Parameter_Category_Name",
         "epiParameter_Estimate_Type",
         "epiParameter_Estimate_Point_Value",
         "epiParameter_Estimate_Unit"]].head()
```

Combining filters

```
params = {
    "Disease_Name_Preferred": "Measles",
    "Estimate_Type_Name": "Case Fatality Ratio (CFR)"
}
response = requests.get(url, params=params)
cfr = pd.DataFrame(response.json())
```

Searching the literature

```
url = "https://collaboratory.who.int/grepi/api/PublicArticle"
params = {"Article_Title": "mpox", "Article_Publication_Year": 2024}
response = requests.get(url, params=params)
articles = pd.DataFrame(response.json())
```

Looking up an article by DOI

```
# Use the DOI without the https://doi.org/ prefix
```

```
params = {"Article_DOI": "10.1093/infdis/jiad091"}

response = requests.get("https://collaboratory.who.int/grepi/api/PublicArticle",
params=params)

article = pd.DataFrame(response.json())
```

Using reference data endpoints

```
# Full disease-pathogen mapping

response =
requests.get("https://collaboratory.who.int/grepi/api/DiseaseCausedbyPathogen")

mapping = pd.DataFrame(response.json())


# Valid disease names

mapping["disease_Name_grEPI_preferred"].unique()
```

Working with nested fields

```
# Nested fields example: population country data

response = requests.get(

    "https://collaboratory.who.int/grepi/api/EpiParameterEstimates",

    params={"Disease_Name_Preferred": "Measles"}

)


# Access country data for the first record

response.json()[0]["epi_Parameter_Population_Country_list"]


# To flatten country data into separate rows:

data = response.json()

data_with_countries = [

    r for r in data

    if r.get("epi_Parameter_Population_Country_list")

]




countries = pd.json_normalize(

    data_with_countries,

    record_path=["epi_Parameter_Population_Country_list"],

    meta=["grEPI_ID"],

    errors="ignore"

)
```

3. Using grEPI Data with epiparameter

The `{epiparameter}` R package (developed by the Epiverse-TRACE initiative) provides a standardized class for working with epidemiological parameters across Epiverse analysis packages and pipelines. Direct integration between grEPI and `{epiparameter}` is planned for the future. In the meantime, you can manually construct `<epiparameter>` objects from grEPI API data.

For full documentation on `{epiparameter}`, see: https://epiverse-trace.github.io/epiparameter/

Step 1: Retrieve a parameter estimate from grEPI

Start by pulling a specific estimate from the API. This example retrieves incubation period estimates for mpox:

```
library(httr2)
library(epiparameter)

# Retrieve incubation period estimates for mpox
resp <- request("https://collaboratory.who.int/grepi/api/EpiParameterEstimates") |>
  req_url_query(
    Disease_Name_Preferred = "Mpox",
    Estimate_Subtype_Name = "Incubation period"
  ) |>
  req_perform()

inc_data <- resp_body_json(resp, simplifyVector = TRUE)

# Look at what is available
inc_data[, c("grEPI_ID", "article_Label",
             "epiParameter_Estimate_Point_Value",
             "epiParameter_Estimate_Unit",
             "epiParameter_Distribution_Type")]

# Select an incubation period estimate record
rec <- inc_data[inc_data$article_Label == "Kroger 2023" &
                inc_data$epiParameter_Distribution_Type == "Log-normal", ][1, ]
```

Step 2: Construct an `<epiparameter>` object

Map grEPI fields to the `epiparameter()` constructor. The level of detail you include depends on what the grEPI record contains. At minimum, you need a disease name, an epidemiological parameter name, and either distribution parameters or summary statistics.

```
# grEPI distribution names use different conventions to R.
# This lookup maps grEPI names to the names accepted by {epiparameter}.
grepi_dist_lookup <- c(
  "Log-normal"  = "lnorm",
  "Gamma"       = "gamma",
  "Weibull"     = "weibull"
)

# Build the <epiparameter> object from grEPI fields.
# This record has a directly reported mean and standard deviation,
# so {epiparameter} can calculate the distribution parameters
# (meanlog/sdlog) automatically.
grepi_eparam <- epiparameter(
  disease = as.character(rec$disease_Name_Preferred),
  pathogen = as.character(rec$pathogen_Species_Name_Preferred),
  epi_name = "incubation_period",
  prob_distribution =
unname(grepi_dist_lookup[rec$epiParameter_Distribution_Type]),
  summary_stats = create_summary_stats(
    mean = as.numeric(rec$epiParameter_Estimate_Point_Value),
    sd = as.numeric(rec$epiParameter_Uncertainty_Single_Value)
  ),
  citation = create_citation(
    author = as.character(rec$article_Authors[[1]]),
    year = as.integer(rec$article_Publication_Year),
    title = as.character(rec$article_Title),
    journal = as.character(rec$literature_Source_Name),
    doi = as.character(rec$article_DOI)
  ),
  metadata = create_metadata(
    units = tolower(gsub(" .*", "", rec$epiParameter_Estimate_Unit)),
    sample_size = as.integer(rec$epi_Parameter_Population_Sample_Size)
```

```
  ),
  notes = paste("grEPI ID:", rec$grEPI_ID)
)

# Inspect the result
grepi_eparam
```

Note: Not all grEPI records contain the same summary statistics. The example above works because the selected record has a directly reported mean and standard deviation. Other records may have different combinations (e.g. mean with a credible interval, or median with a range). Before attempting conversion, check what the record contains in the `epiParameter_Estimate_Value_Type`, `epiParameter_Uncertainty_Value_Type`, and `epiParameter_Uncertainty_Single_Value` fields. Records without sufficient summary statistics for `{epiparameter}` to calculate distribution parameters can still be stored as unparameterised `<epiparameter>` objects by omitting the `prob_distribution` argument. Refer to `{epiparameter}` documentation for further information: https://epiverse-trace.github.io/epiparameter/

Step 3: Use in an analysis pipeline

Once you have an `<epiparameter>` object, it works with the standard `{epiparameter}` functions and can be passed to other Epiverse-TRACE packages. For example, you can generate random draws from the distribution, calculate density values, or use it as an input to transmission models:

```
# Generate random draws from the distribution
generate(grepi_eparam, times = 10)

# Calculate density, CDF, and quantiles
density(grepi_eparam, at = 5)
cdf(grepi_eparam, q = 7)
quantile(grepi_eparam, p = c(0.025, 0.5, 0.975))

# Plot the distribution
plot(grepi_eparam)
```

Converting multiple records

To convert a batch of grEPI records, wrap the construction logic in a function and apply it across rows. This is useful when you want to compare multiple estimates for the same parameter:

```
grepi_to_epiparameter <- function(rec, dist_lookup) {
  # Only convert records with a point estimate and supported distribution
  if (is.na(rec$epiParameter_Estimate_Point_Value)) return(NULL)

  # Map distribution name if available
  r_dist <- if (!is.na(rec$epiParameter_Distribution_Type) &&
                rec$epiParameter_Distribution_Type %in% names(dist_lookup)) {
    unname(dist_lookup[rec$epiParameter_Distribution_Type])
  } else {
    NA_character_
  }

  # Determine SD: use directly reported value if available,
  # otherwise skip
  sd_val <- if (!is.na(rec$epiParameter_Uncertainty_Value_Type) &&
                rec$epiParameter_Uncertainty_Value_Type == "SD (Standard
Deviation)" &&
                !is.na(rec$epiParameter_Uncertainty_Single_Value)) {
    as.numeric(rec$epiParameter_Uncertainty_Single_Value)
  } else {
    NA_real_
  }

  tryCatch(
    epiparameter(
      disease = as.character(rec$disease_Name_Preferred),
      pathogen = as.character(rec$pathogen_Species_Name_Preferred),
      epi_name = "incubation_period",
      prob_distribution = r_dist,
      summary_stats = create_summary_stats(
        mean = if (rec$epiParameter_Estimate_Value_Type == "Mean")
                 as.numeric(rec$epiParameter_Estimate_Point_Value) else NA_real_,
        median = if (rec$epiParameter_Estimate_Value_Type == "Median")
```

```
                    as.numeric(rec$epiParameter_Estimate_Point_Value) else NA_real_,
        sd = sd_val
      ),
      citation = create_citation(
        author = as.character(rec$article_Authors[[1]]),
        year = as.integer(rec$article_Publication_Year),
        title = as.character(rec$article_Title),
        journal = as.character(rec$literature_Source_Name),
        doi = as.character(rec$article_DOI)
      ),
      metadata = create_metadata(
        units = tolower(gsub(" .*", "", rec$epiParameter_Estimate_Unit)),
        sample_size = as.integer(rec$epi_Parameter_Population_Sample_Size)
      )
    ),
    error = function(e) {
      message("Skipping record ", rec$grEPI_ID, ": ", e$message)
      NULL
    }
  )
}

# Apply across all incubation period records
eparam_list <- lapply(seq_len(nrow(inc_data)), function(i) {
  grepi_to_epiparameter(inc_data[i, ], grepi_dist_lookup)
})

# Remove NULLs (records that could not be converted)
eparam_list <- Filter(Negate(is.null), eparam_list)
cat("Successfully converted", length(eparam_list), "of", nrow(inc_data),
"records\n")
```

4. Endpoint Reference

This section provides the full technical detail for each endpoint: query parameters, response field names, and data types. Fields are grouped by their naming prefix for readability.

4.1 EpiParameterEstimates

GET `https://collaboratory.who.int/grepi/api/EpiParameterEstimates`

Query parameters

Parameter	Type	Description
`Disease_Name_Preferred`	string	Filter by disease name (e.g. "Measles", "Mpox")
`Pathogen_Species_Name_Preferred`	string	Filter by pathogen species (e.g. "Measles virus")
`Pathogen_Strain_Preferred`	string	Filter by pathogen strain
`Pathogen_Sub_Strain_Preferred`	string	Filter by pathogen sub-strain
`Estimate_Type_Name`	string	Filter by estimate type (e.g. "Case Fatality Ratio (CFR)")
`Estimate_Subtype_Name`	string	Filter by estimate subtype
`Time_From`	string	Filter by event type for the start of the interval (e.g., “Exposure”, “Infection”, “Symptom Onset”)
`Time_To`	string	Filter by event type for the end of the interval (e.g., “Symptom Onset”, “Reporting”, “Recovery”)
`Created_Date_From`	datetime	Records created on or after this date (YYYY-MM-DDTHH:MM:SS)
`Created_Date_To`	datetime	Records created on or before this date

Response fields

Over 140 fields are returned per record. The main field groups, by naming prefix, are:

Identification: `grEPI_ID` (UUID).

disease_* fields: `disease_Name_Preferred`, `disease_Code_ICD`, `disease_Sub_Chapter_Name_ICD`, `disease_Chapter_Name_ICD`, `disease_Chapter_Code_ICD`, `disease_Spread_Type`.

pathogen_* fields: `pathogen_Species_Name_Preferred`, `pathogen_Species_Code_ICD`, `pathogen_Genus_Name_Preferred`, `pathogen_Family_Name_Preferred`.

epiParameter_Estimate_* fields: `epiParameter_Estimate_Type`, `epiParameter_Estimate_Subtype`, `epiParameter_Estimate_Value_Type` (e.g. "Median", "Crude proportion"), `epiParameter_Estimate_Point_Value` (numeric), `epiParameter_Estimate_Range_Lower_Bound_Value`, `epiParameter_Estimate_Range_Upper_Bound_Value`, `epiParameter_Estimate_Unit`. For delay parameters: `epiParameter_EventFrom` and `epiParameter_EventTo`.

epi_Parameter_Category_Name: the parameter category (e.g. "Severity", "Delays").

epiParameter_Uncertainty_* fields: `epiParameter_Uncertainty_Value_Type`, `epiParameter_Uncertainty_Single_Value`, `epiParameter_Uncertainty_Range_Lower_Bound_Value`, `epiParameter_Uncertainty_Range_Upper_Bound_Value`, `epiParameter_Uncertainty_Unit`.

epiParameter_Distribution_* fields: `epiParameter_Distribution_Type`. Up to three distribution parameters, each with value, value type, and uncertainty fields, following the pattern `epiParameter_Distribution_Parameter1_Value` through `epiParameter_Distribution_Parameter3_Value`.

epi_Parameter_Method_* fields: `epi_Parameter_Method_PercentageUnit_Calculation` (e.g. "Naive"), `epi_Parameter_Method_Inference`, and related boolean fields for censoring, truncation, bias adjustment, and discretization.

epi_Parameter_PercentageUnit_* fields: for proportion-based estimates: `epi_Parameter_PercentageUnit_numerator`, `epi_Parameter_PercentageUnit_denominator`.

epi_Parameter_CaseClassification_* fields: `epi_Parameter_CaseClassification_isConfirmed`, `epi_Parameter_CaseClassification_isProbable`, `epi_Parameter_CaseClassification_isSuspected`, `epi_Parameter_CaseClassification_isUnspecified` (all boolean).

epi_Parameter_Population_* fields: `epi_Parameter_Population_Sample_Size` (integer), age fields (min/max in years and months), `epi_Parameter_Population_Sex_List` (nested array), `epi_Parameter_Population_Setting`, `epi_Parameter_Population_Group`, `epi_Parameter_Population_Comment` (free text). Study period: `epi_Parameter_Population_Study_Start_Year` / `epi_Parameter_Population_Study_End_Year`. `epi_Parameter_Population_Country_list` is a nested array of objects, each containing

`country_Name`, `country_Code_ISO2`, `country_Code_ISO3`, `country_World_Continent`, `country_WHO_Region`.

article_* fields: `article_Title`, `article_DOI` (e.g. "10.1016/j.vaccine.2022.02.057"), `article_Label` (short citation key, e.g. "Bose 2022"), `article_Publication_Year` (integer), `literature_Source_Name`, `article_Authors` (flat array of name strings).

epi_Parameter_DataSource_* fields:
`epi_Parameter_Data_ExtractedBy_OrganizationGroup_Name`, `epi_Parameter_DataSource_Primary_ImportedFrom_Name`, `epi_Parameter_DataSource_Primary_ImportedFrom_Project`, `epi_Parameter_DataSource_Name`, `epiParameter_CreatedDate` (datetime).

4.2 PublicArticle

GET `https://collaboratory.who.int/grepi/api/PublicArticle`

Returns article records from the grEPI literature database.

Query parameters

Parameter	Type	Description
`Article_DOI`	string	Filter by DOI (e.g. "10.1093/infdis/jiad091")
`Article_Title`	string	Filter by article title (partial, case-insensitive match)
`Article_Label`	string	Filter by short citation key (e.g. "Miura 2024")
`Article_Publication_Year`	integer	Filter by publication year (e.g. 2024)
`Author`	string	Filter by author name
`Literature_Source_Name`	string	Filter by journal or source name (e.g. "Lancet")
`Literature_Source_Type_Name`	string	Filter by source type (e.g. "Journal")

Response fields

`article_ID` (UUID), `article_DOI`, `article_Title`, `article_Label`, `article_Publication_Year` (integer), `article_Paper_Version_Only` (boolean), `article_Page_First` / `article_Page_Last` (integer), `literature_Source_Name`, `literature_Source_Type_Name`, `article_grEPI_RecordStatus`, `article_grEPI_LastModifiedDate` (datetime).

`article_Authors` is a nested array of objects containing: `article_Author_ID` (UUID), `person_ID` (UUID), `person_First_Names`, `person_Last_Names`, `authorOrder` (integer). Note: on the `EpiParameterEstimates` endpoint, `article_Authors` is returned as a flat array of name strings rather than structured objects.

4.3 PathogenTaxonomy

GET `https://collaboratory.who.int/grepi/api/PathogenTaxonomy`

Returns pathogen taxonomy records with both grEPI preferred names and ICD classification codes.

Query parameters

Parameter	Type	Description
`Pathogen_Type_Name_ICD`	string	Filter by pathogen type (ICD name, e.g. "Virus")

`Pathogen_Family_Name_grEPI_preferred`	string	Filter by family (e.g. "Poxvirus")
`Pathogen_Genus_Name_grEPI_preferred`	string	Filter by genus (e.g. "Orthopoxvirus")
`Pathogen_Species_Name_grEPI_preferred`	string	Filter by species (e.g. "Monkeypox virus")
`Pathogen_Strain_Name_grEPI_preferred`	string	Filter by strain
`Pathogen_SubStrain_Name_grEPI_preferred`	string	Filter by sub-strain
`Last_Modified_Date_From`	datetime	Records modified on or after this date (YYYY-MM-DDTHH:MM:SS)
`Last_Modified_Date_To`	datetime	Records modified on or before this date

Response fields

`pathogen_Species_ID` (UUID). For each taxonomic level (Type, Family, Genus, Species, Strain, SubStrain), four fields are returned: `pathogen_[Level]_Name_grEPI_preferred` (the grEPI name), `pathogen_[Level]_Name_ICD` (the ICD name), `pathogen_[Level]_Code_ICD` (the ICD code), and `pathogen_[Level]_Identifier_ICD` (the ICD identifier). For example: `pathogen_Family_Name_grEPI_preferred, pathogen_Family_Name_ICD, pathogen_Family_Code_ICD, pathogen_Family_Identifier_ICD`. Also includes `diseaseCausedByPathogen_grEPI_RecordStatus` and `diseaseCausedByPathogen_grEPI_LastModifiedDate`.

4.4 DiseaseCausedbyPathogen

GET `https://collaboratory.who.int/grepi/api/DiseaseCausedbyPathogen`

Returns the full mapping between diseases and their causative pathogens.

Query parameters

Parameter	Type	Description
`Last_Modified_Date_From`	datetime	Records modified on or after this date (YYYY-MM-DDTHH:MM:SS)
`Last_Modified_Date_To`	datetime	Records modified on or before this date

Response fields

`disease_Pathogen_ID` (UUID). Disease fields: `disease_Name_grEPI_preferred`, `disease_Name_ICD`, `disease_Code_ICD`, `disease_Identifier_ICD`, plus sub-chapter and chapter fields. Pathogen fields follow the same four-field-per-level pattern described for `PathogenTaxonomy` above. Also includes `diseaseCausedByPathogen_grEPI_RecordStatus` and `diseaseCausedByPathogen_grEPI_LastModifiedDate`.